

Python数据容器



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



目录

Contents

- ◆ 字典的定义及其应用场景
- ◆ 集合的定义及其应用场景
- ◆ 数据序列的公共方法
- ◆ 列表集合字典推导式

学习目标

Learning Objectives

1. 知道字典的应用场景
2. 掌握字典的基本使用



字典的定义及其应用场景

什么是字典

生活中的字典



开发中的字典

```
info = {'name': '班长', 'id': 100, 'sex': 'f', 'address': '地球亚洲中国北京'}
```

说明:

- 字典和列表一样，也能够存储多个数据
- 列表中找到某个元素时，是根据下标进行的
- 字典中找到某个元素时，是根据‘名字’（就是冒号:前面的那个值，例如上面代码中的‘name’、‘id’、‘sex’）
- 字典的每个元素由2部分组成，键:值。例如 ‘name’:‘班长’，‘name’为键，‘班长’为值

为什么需要字典

思考1：如果有多个数据，例如：'Tom'，'男'，20，如何快速存储？

答：列表

```
list1 = ['Tom', '男', 20]
```

思考2：如何查找到数据'Tom'？

答：查找到下标为0的数据即可。

```
list1[0]
```

思考3：如果将来数据顺序发生变化，如下所示，还能用`list1[0]`访问到数据'Tom'吗？

```
list1 = ['男', 20, 'Tom']
```

答：不能，数据'Tom'此时下标为2。

思考4：数据顺序发生变化，每个数据的下标也会随之变化，如何保证数据顺序变化前后能使用同一的标准查找数据呢？

答：字典，字典里面的数据是以**键值对**形式出现，字典数据和数据顺序没有关系，即字典不支持下标，后期无论数据如何变化，只需要按照对应的键的名字查找数据即可。

字典的定义

字典特点：

- ❑ 符号为大括号（花括号）
- ❑ 数据为键值对形式出现
- ❑ 各个键值对之间用逗号隔开

基本语法：

```
# 有数据字典
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}

# 空字典
dict2 = {}

dict3 = dict()
```

注意事项

注意：一般称冒号前面的为键(key)，简称k；冒号后面的为值(value)，简称v；key:value就是我们通常说的键值对了。

字典的“增”操作

基本语法：

```
字典序列[key] = value
```

注：如果key存在则修改这个key对应的值；如果key不存在则新增此键值对。

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}

dict1['name'] = 'Rose'
# 结果: {'name': 'Rose', 'age': 20, 'gender': 'male'}
print(dict1)

dict1['id'] = 110

# {'name': 'Rose', 'age': 20, 'gender': 'male', 'id': 110}
print(dict1)
```

注意事项

注意：字典为可变类型

字典的“删”操作

`del()` / `del`: 删除字典或删除字典中指定键值对

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
  
del dict1['gender']  
  
# 结果: {'name': 'Tom', 'age': 20}  
print(dict1)
```

`clear()`: 清空字典

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
  
dict1.clear()  
print(dict1)  # {}
```

字典的“改”操作

基本语法:

```
字典序列[key] = value
```

注意事项

注意：如果key存在则修改这个key对应的值；如果key不存在则新增此键值对。

字典的“查”操作

key值查找

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
  
print(dict1['id'])          # 报错  
print(dict1['name'])       # Tom
```

说明：如果当前查找的key存在，则返回对应的值；否则则报错。

编号	函数	作用
1	get(key, 默认值)	根据字典的key获取对应的value值，如果当前查找的key不存在则返回第二个参数(默认值)，如果省略第二个参数，则返回None
2	keys()	以列表返回一个字典所有的键
3	values()	以列表返回字典中的所有值
4	items()	以列表返回可遍历的(键，值)元组数组

字典的“查”操作

get方法演示

字典序列.get(key, 默认值)

说明：如果当前查找的key不存在则返回第二个参数(默认值)，如果省略第二个参数，则返回None。

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}
```

```
print(dict1.get('name'))    # Tom
```

```
print(dict1.get('id', 110)) # 110
```

```
print(dict1.get('id'))      # None
```

字典的“查”操作

keys()、values()、items() 案例演示

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
print(dict1.keys())
```

```
# 结果: dict_keys(['name', 'age', 'gender'])
```

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
print(dict1.values())
```

```
# 结果: dict_values(['Tom', 20, 'male'])
```

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
print(dict1.items())
```

```
# 结果: dict_items([('name', 'Tom'), ('age', 20), ('gender', 'male')])
```

案例演示

遍历字典的key

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
for key in dict1.keys():  
    print(key)
```

遍历字典的value

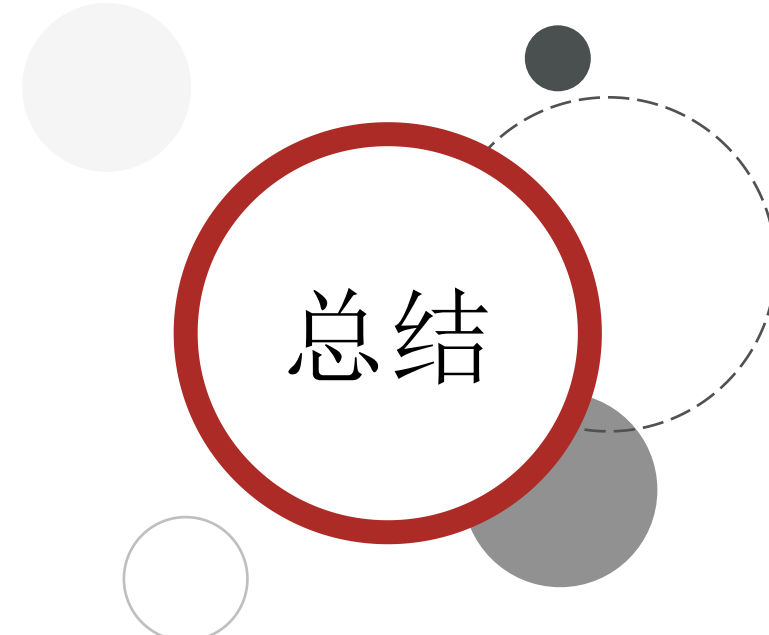
```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
for value in dict1.values():  
    print(value)
```

遍历字典的元素

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
for item in dict1.items():  
    print(item)
```

遍历字典的键值对

```
dict1 = {'name': 'Tom', 'age': 20, 'gender': 'male'}  
for key, value in dict1.items():  
    print(f'{key} = {value}')
```



总结

1. 定义字典

```
dict1 = {'name': 'Python', 'age': 30}  
dict2 = {}  
dict3 = dict()
```

2. 常见操作

- 增/改

字典序列[key] = 值

- 查找

字典序列[key]



目录

Contents

- ◆ 字典的定义及其应用场景
- ◆ 集合
- ◆ 数据序列的公共方法
- ◆ 列表集合字典推导式

学习目标

Learning Objectives

1. 知道集合的特点
2. 掌握的基本使用



集合

什么是集合

对于日常编程的使用中，集合有一个很重要的特性需要知道

集合（set）是一个不重复无序的元素序列

重复一遍：集合里边的元素是不可重复的，无序的

集合的主要功能在于去重操作

集合的定义

基本语法:

创建集合使用`{}`或`set()`， 但是如果要创建空集合只能使用`set()`， 因为`{}`用来创建空字典。

```
s1 = {10, 20, 30, 40, 50}
print(s1)

s2 = {10, 30, 20, 10, 30, 40, 30, 50}
print(s2)

s3 = set(' abcdefg')
print(s3)

s4 = set()
print(type(s4))  # set

s5 = {}
print(type(s5))  # dict
```

集合的使用

☆ 去重

当一个列表中的元素有重复值的时候，我们可以使用集合进行去重

```
my_list = ['a', 'b', 1, 'a', 'b']  
my_set = set(my_list) # 列表转集合  
print(my_set)  
  
my_list = list(my_set) # 集合转列表  
print(my_list)
```

集合本身是无序的，去重之后无法保留原来的顺序



目录

Contents

- ◆ 字典的定义及其应用场景
- ◆ 集合
- ◆ 数据序列的公共方法
- ◆ 列表集合字典推导式

学习目标

Learning Objectives

1. 掌握数据容器的公共运算符
2. 掌握数据容器的公共方法



数据容器的公共方法

方法介绍

运算符	描述	支持的容器类型
+	合并	字符串、列表、元组
*	复制	字符串、列表、元组
in	元素是否存在	字符串、列表、元组、字典
not in	元素是否不存在	字符串、列表、元组、字典

+ 合并操作

```
# 1. 字符串
str1 = 'aa'
str2 = 'bb'
str3 = str1 + str2
print(str3) # aabb
```

```
# 2. 列表
list1 = [1, 2]
list2 = [10, 20]
list3 = list1 + list2
print(list3) # [1, 2, 10, 20]
```

```
# 3. 元组
t1 = (1, 2)
t2 = (10, 20)
t3 = t1 + t2
print(t3) # (1, 2, 10, 20)
```

* 复制操作

```
# 1. 字符串
print('-' * 10)

# 2. 列表
list1 = ['hello']
print(list1 * 4)

# 3. 元组
t1 = ('world',)
print(t1 * 4)
```



```
Run: Python公共方法 x
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python公共方法.py
-----
['hello', 'hello', 'hello', 'hello']
('world', 'world', 'world', 'world')

Process finished with exit code 0
```

in与not in

1. 字符串

```
print('a' in 'abcd') # True  
print('a' not in 'abcd') # False  
print('-' * 10)
```

2. 列表

```
list1 = ['a', 'b', 'c', 'd']  
print('a' in list1) # True  
print('a' not in list1) # False  
print('-' * 10)
```

3. 元组

```
t1 = ('a', 'b', 'c', 'd')  
print('aa' in t1) # False  
print('aa' not in t1) # True  
print('-' * 10)
```

4. 字典

```
dict1 = {'name': 'heima', 'age': 18}  
print('name' in dict1) # True  
print('age' not in dict1) # False
```



数据序列的公共方法

公共方法

编号	函数	描述
1	len()	计算容器中元素个数
2	del 或 del()	删除
3	max()	返回容器中元素最大值
4	min()	返回容器中元素最小值
5	range(start, end, step)	生成从start到end的数字，步长为 step，供for循环使用
6	enumerate()	函数用于将一个可遍历的数据对象(如列表、元组或字符串)组合为一个索引序列，同时列出数据和数据下标，一般用在 for 循环当中。

len() 方法

```
# 1. 字符串
str1 = 'abcdefg'
print(len(str1)) # 7

# 2. 列表
list1 = [10, 20, 30, 40]
print(len(list1)) # 4

# 3. 元组
t1 = (10, 20, 30, 40, 50)
print(len(t1)) # 5

# 4. 集合
s1 = {10, 20, 30}
print(len(s1)) # 3

# 5. 字典
dict1 = {'name': 'Rose', 'age': 18}
print(len(dict1)) # 2
```

del() 方法

1. 字符串

```
str1 = 'abcdefg'
```

```
del str1
```

```
print(str1)
```

2. 列表

```
list1 = [10, 20, 30, 40]
```

```
del(list1[0])
```

```
print(list1) # [20, 30, 40]
```


max() 方法

1. 字符串

```
str1 = 'abcdefg'  
print(max(str1)) # g
```

2. 列表

```
list1 = [10, 20, 30, 40]  
print(max(list1)) # 40
```

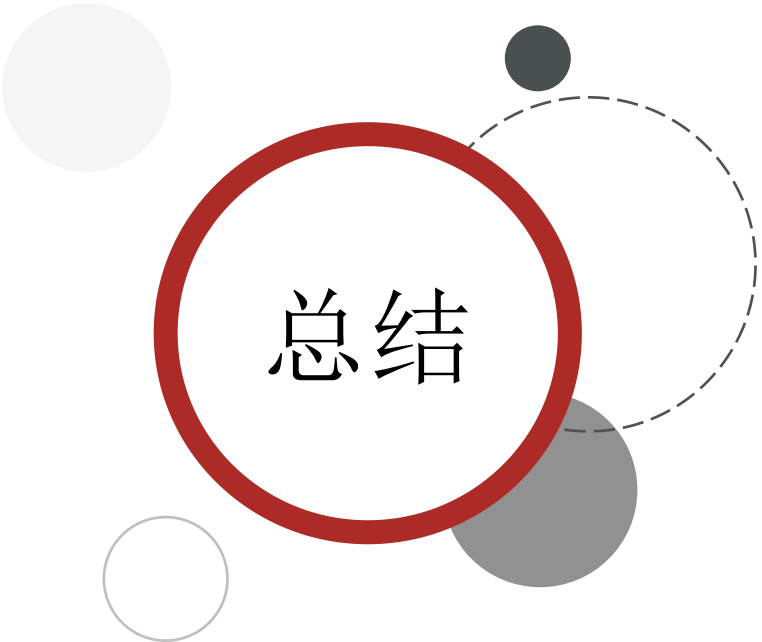
min() 方法

1. 字符串

```
str1 = 'abcdefg'  
print(min(str1)) # a
```

2. 列表

```
list1 = [10, 20, 30, 40]  
print(min(list1)) # 10
```



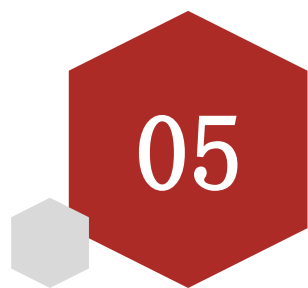
总结

1. 运算符

- +
- in / not in

2. 公共方法

- len()
- del()



列表集合字典推导式

什么是推导式

推导式 `comprehensions`（又称解析式），是Python的一种独有特性。推导式是可以从一个数据序列构建另一个新的数据序列（一个有规律的列表或控制一个有规律列表）的结构体

共有三种推导：

- ①列表推导式
- ②集合推导式
- ③字典推导式

为什么需要列表推导式

案例：创建一个0-9的列表

while

```
# 1. 准备一个空列表
list1 = []

# 2. 书写循环，依次追加数字到空列表list1中
i = 0
while i < 10:
    list1.append(i)
    i += 1

print(list1)
```

for

```
list1 = []
for i in range(10):
    list1.append(i)

print(list1)
```

列表推导式

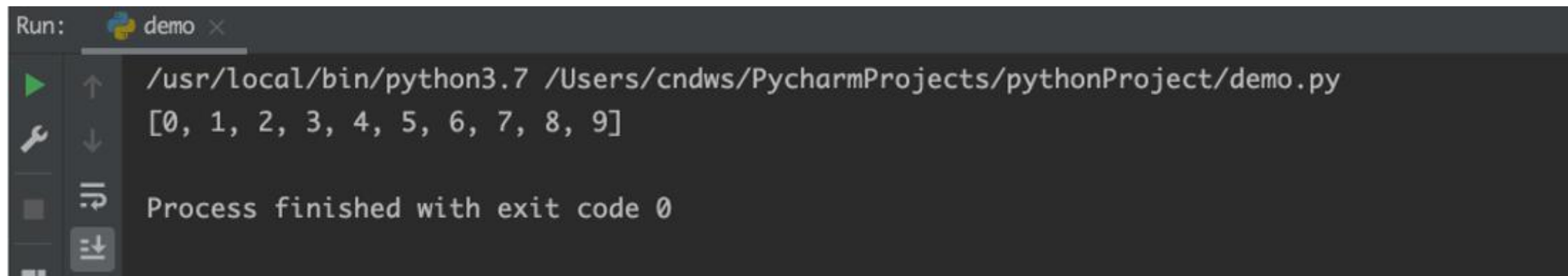
基本语法:

```
变量名 = [表达式 for 变量 in 列表 for 变量 in 列表]  
变量名 = [表达式 for 变量 in 列表 if 条件]
```

案例：创建一个0-9的列表

```
list1 = [i for i in range(10)]  
print(list1)
```

运行结果:



```
Run: demo x  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
Process finished with exit code 0
```

带if的列表推导式

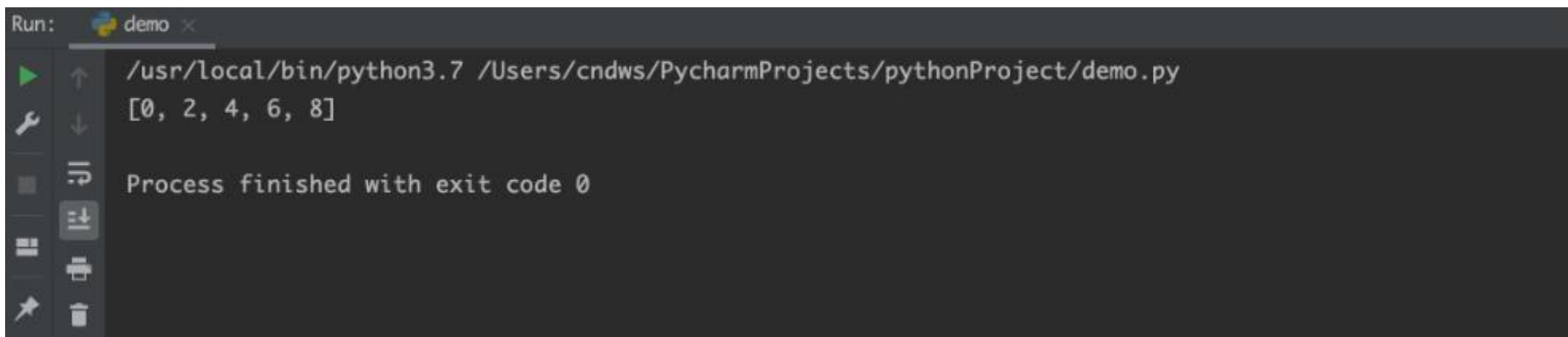
案例：创建一个0-9的偶数列表

方法一：range() 步长实现

```
list1 = [i for i in range(0, 10, 2)]  
print(list1)
```

方法二：if实现

```
list1 = [i for i in range(10) if i % 2 == 0]  
print(list1)
```

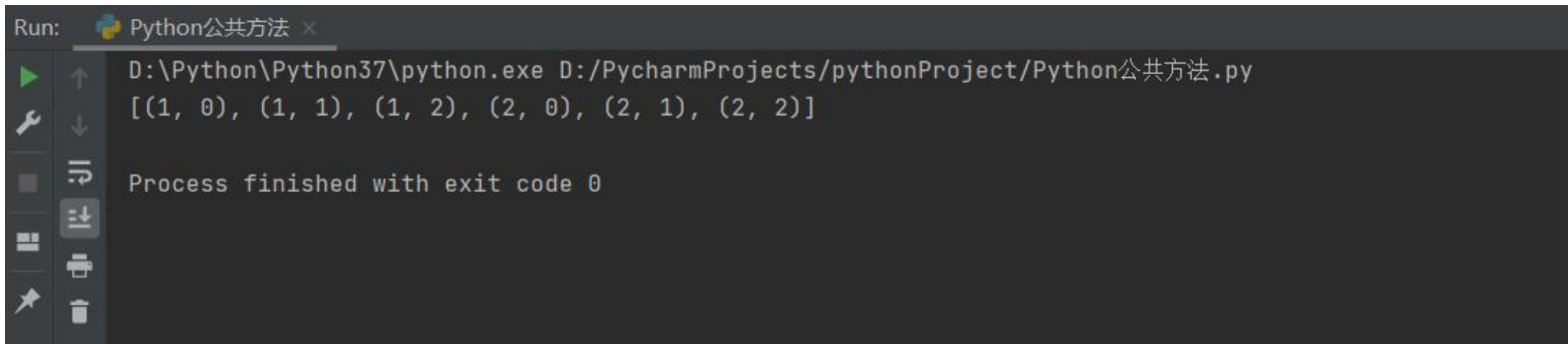


The screenshot shows the PyCharm Run console for a file named 'demo.py'. The command executed is `/usr/local/bin/python3.7 /Users/cndws/PycharmProjects/pythonProject/demo.py`. The output is `[0, 2, 4, 6, 8]`. Below the output, it says 'Process finished with exit code 0'. The left sidebar of the Run window contains various icons for debugging and execution control.

多个for循环实现列表推导式

案例：创建列表 => [(1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]

```
list1 = [(i, j) for i in range(1, 3) for j in range(3)]  
print(list1)
```



```
Run: Python公共方法 ×  
D:\Python\Python37\python.exe D:/PycharmProjects/pythonProject/Python公共方法.py  
[(1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]  
Process finished with exit code 0
```


字典推导式

思考：有如下两个列表，如何快速合并为一个字典？

```
list1 = ['name', 'age', 'gender']
```

```
list2 = ['Tom', 20, 'man']
```

答：使用字典推导式

基本语法：

- 字典推导式列表推导式思想的延续，语法差不多，只不过产生的是字典而已。
- 字典推导式格式：
- 变量名 = {...}

字典推导式作用：快速合并列表为字典或提取字典中目标数据。

字典推导式

案例1：创建一个字典：字典key是1-5数字，value是这个数字的2次方。

```
dict1 = {i: i**2 for i in range(1, 5)}  
print(dict1) # {1: 1, 2: 4, 3: 9, 4: 16}
```

案例2：将两个列表合并为一个字典

```
list1 = ['name', 'age', 'gender']  
list2 = ['Tom', 20, 'man']  
  
dict1 = {list1[i]: list2[i] for i in range(len(list1))}  
print(dict1)
```

集合推导式

集合推导式跟列表推导式非常相似，唯一区别在于用 { } 代替 []

需求：创建一个集合，数据为下方列表的2次方。

```
list1 = [1, 1, 2]
```

代码：

```
list1 = [1, 1, 2]
set1 = {i ** 2 for i in list1}
print(set1) # {1, 4}
```

注意事项

注意：集合有数据去重功能。



传智教育旗下高端IT教育品牌